Design and Analysis of Algorithms
Taubakabyl Nurlybek
Assignment 3 - Graphs: DSU, MST, SCC and Shortest Paths
Deadline: 23.10.2026 23:59

Submission: ZIP archive named DAA_Assignment3_name_surname_group.zip

- Upload to Moodle.
- Include a link to your GitHub repository (branch main, tag v1.0).
- **Defense:** Practice classes in Week 9. You will be asked to run your tests and benchmark, explain your code, justify your complexity table, and say which algorithm you would pick for a given situation and why.

Scenario: City Network & Job Scheduler

The city platform you have been building now works with networks. The road department asks which roads to build so that every district is reachable at the lowest total cost. The scheduler asks in which order dependent jobs may run, which jobs form a deadlock, and what the fastest route through the pipeline is.

All of these are graph questions, and the standard library has no graph types. Your team writes its own, counts the work each algorithm does, and decides which algorithm fits which situation.

In **Assignment 3**, you will:

- Implement **DSU**, **Kruskal**, **Prim**, **Kosaraju**, **Kahn** and **shortest paths in a DAG**.
- Prove your implementations are correct by cross-checking them against each other and against brute force.
- Count the operations each algorithm performs and compare the counts with the complexity formulas.
- Write down which algorithm you would choose for which kind of graph, and why.

Learning Objectives

- Implement **DSU** with union by rank (or size) and path compression, and explain the almost-constant α(n) cost.
- Build a **minimum spanning tree** in two different ways and show that both give the same total weight.
- Find **strongly connected components** and a **topological order** with an iterative DFS that survives 100 000 vertices.
- Compute **shortest paths in a DAG** in O(V + E) and explain why negative weights are allowed there.
- Validate complexity **by counting operations**, which is reproducible on any machine, unlike wall-clock time.
- Choose the right algorithm for a given graph and justify the choice.
- Keep a clean **Git history** with feature branches and a release tag.

System Requirements

1. Graph Core

- **Representation:** Adjacency list. Vertices are numbered 0 .. n-1, weights are int. One

class must support both directed and undirected graphs.

- **Input Format:** A text file: the first line is n m directed|undirected, then m lines u v w. Put at least three small graphs in data/ (one disconnected, one cyclic) and use them in the tests.
- **Edge Cases:** Self-loops, parallel edges, isolated vertices and disconnected graphs must not crash anything.
- **Libraries:** Unlike Assignment 2, java.util collections are allowed here (ArrayList, PriorityQueue, ArrayDeque), but every algorithm must be written by you.

2. Disjoint Set Union

- **Operations:** makeSet(n), find(x), union(x, y), components().
- **Both Heuristics:** Union by rank or by size **and** path compression. One without the other is not enough.
- **Counters:** Count find calls and successful union calls.

3. MST: Kruskal & Prim

- **Kruskal:** Sort the edges by weight, add an edge when find(u) != find(v), then union(u, v).
- **Prim:** Start from vertex 0 and grow the tree with a PriorityQueue, as on the lecture slide. Skip a vertex that is already in the tree.
- **Disconnected Graph:** Return a minimum spanning **forest** and report the number of components. Do not throw.
- **Agreement:** On the same graph both algorithms must return the same total weight (the trees themselves may differ). Your benchmark must check this automatically and fail loudly if it breaks.

4. SCC & Topological Ordering

- **Kosaraju:** First DFS pass to get the finish order, reverse the graph, second pass in decreasing finish order - each tree is one component.
- **Iterative DFS is required:** at n = 100 000 a recursive DFS overflows the JVM stack. Use your own explicit stack and record its maximum depth in the metrics.
- **Kahn:** In-degrees plus a queue. If fewer than n vertices leave the queue, the graph has a **cycle** - report it instead of returning a wrong order.
- **Validation:** A returned order must satisfy pos[u] < pos[v] for every edge u -> v.

5. Shortest Paths in DAGs

- **Algorithm:** Take the topological order, then relax the outgoing edges of every vertex in that order - one pass, O(V + E).
- **Sources:** Every vertex with in-degree 0 starts at distance 0 (a virtual super-source), so the whole DAG is reachable and the result does not depend on which vertex you happened to pick.
- **Negative Weights:** Allowed here. Explain in the report why this is safe in a DAG but not in a general graph.
- **Path Restoring:** Keep a parent[] array so that the actual path to a vertex can be printed.

6. Benchmark & Metrics

Wall-clock time depends on the laptop, the JVM version and what else is running, so **the graded evidence is the operation counters**: with the generator below they are the same on every machine.

- **Counters:** dsuOps (find + union), edgeScans (edges looked at), comparisons (weights compared) and maxDepth (deepest your explicit stack ever got). Count them inside the algorithms.
- **Ten Graphs:** five sizes, each in two densities. The sizes go in half-decade steps, so every point is the previous one times the square root of ten: n = 1 000, 3 162, 10 000, 31 623, 100 000. Densities: sparse with m = 4n and dense with m = 20n.
- **Three Graph Kinds** per size and density: an undirected graph for MST, a directed graph for Kosaraju, and a DAG for Kahn and the shortest paths.
- **Exact Generator** (so that your numbers match the reference table below) - always new Random(42):
  - **undirected:** shuffle perm = [0..n-1] with for (int i = n-1; i > 0; i--) swap(perm[i], perm[rnd.nextInt(i+1)]); then for i = 1..n-1 add the edge (perm[rnd.nextInt(i)], perm[i]) - this is a random spanning tree, so the graph is connected; then add the remaining edges as a = rnd.nextInt(n), b = rnd.nextInt(n) with b != a;
  - **directed:** only the random edges, no spanning tree;
  - **DAG:** shuffle rank[0..n-1] the same way, then for every random edge swap the ends if rank[a] > rank[b];
  - every weight is 1 + rnd.nextInt(1000).
- **Runs:** Measure time as the **median of 5 runs** and put it in the CSV, but draw your conclusions from the counters, not from the clock. On the two smallest graphs the clock is mostly noise - that is expected, they are there for the counters.
- **CSV:** results.csv with one row per algorithm and graph (50 rows in total), columns algorithm,graph,n,m,time_ms,dsu_ops,edge_scans,comparisons,max_depth,result. The result column holds the MST weight (Kruskal, Prim), the number of components (Kosaraju), the number of vertices in the order (Kahn) or the sum of all distances (DagShortestPath).

7. Reference Values (check yourself)

Your program must print exactly these numbers. They do not depend on the machine, on the JDK version or on the order of your adjacency lists - if one of them differs, the algorithm has a bug.

Graph (n / m)	MST total weight	SCC count	Sum of DAG distances
1 000 / 4 000	147 970	27	396 274
1 000 / 20 000	30 363	1	252 411
3 162 / 12 648	468 911	137	1 271 911
3 162 / 63 240	97 589	1	723 157
10 000 / 40 000	1 486 139	373	3 941 828
10 000 / 200 000	309 550	1	2 381 342
31 623 / 126 492	4 715 515	1 188	12 551 891
31 623 / 632 460	968 517	1	7 376 612
100 000 / 400 000	15 012 094	3 923	40 096 539
100 000 / 2 000 000	3 056 681	1	23 512 087

Two counters follow directly from the algorithm and must match exactly at every size and density: Kosaraju scans **2m** edges (two passes), Kahn and the DAG relaxation scan **m** each.

8. Tests (JUnit 5)

- **DSU:** After random unions, find must agree with a simple reference implementation, and components() must match the number of sets.
- **MST:** On 100 random connected graphs Kruskal and Prim return the same total weight, and the tree has exactly n - 1 edges.
- **SCC:** Compare with brute force (BFS from every vertex, both directions) on random graphs with n ≤ 200.
- **Topological Order:** Valid order on a DAG, and a cycle is detected on a cyclic graph.
- **DAG Shortest Paths:** Compare with Bellman-Ford on small random DAGs, including negative weights.
- **Stack Safety:** SCC on a path of 100 000 vertices must finish without StackOverflowError.

9. Report (REPORT.md or PDF, 3-5 pages)

- **Complexity Table:** Time and extra space for DSU find/union, Kruskal, Prim, Kosaraju, Kahn and DAG shortest paths, with one line of justification per row.
- **Counters vs Formulas:** For each algorithm, divide your measured counter by the formula (for example edgeScans / (V + E) for Kosaraju, or dsuOps / E for Kruskal) and show that the result stays constant as n grows. A constant ratio is the empirical

confirmation of the complexity class.
- **Correctness Arguments** (5-8 sentences each):
  - **Cut property** - why the lightest edge across a cut is safe, and how Kruskal and Prim both use it;
  - **Induction on the topological order** - why a single pass of relaxations is enough in a DAG.
- **Which Algorithm When:** A short table - sparse vs dense graph, edge list vs adjacency list, cycle possible or not - saying which algorithm you would use and why. This is the main question at the defense.
- **Slope Check:** On log-log axes a straight line of slope 1 means linear work. Fit the slope of your edgeScans curves: the traversals give 1.00, while Kruskal is slightly steeper because of the log factor in E log E. Report both numbers.
- **Plots (three):**
  - **Operations vs n** - one line per algorithm and density, log-log axes;
  - **Scans per edge** - edgeScans / m vs n; a flat line is the confirmation of linear work;
  - **Share of edges Kruskal examines** - edgeScans / m for Kruskal on sparse and on dense graphs. This one explains where each MST algorithm wins.
- **One Paragraph on Time:** What your measured times looked like, and why they are not the main evidence (other hardware, other JVM, background load).

10. Git Workflow

- **Branches:** main (only working code, tag v1.0) and feature/dsu, feature/mst, feature/scc, feature/paths, feature/metrics.
- **Commits:** Small steps with clear messages, e.g. feat(dsu): add path compression, test(mst): compare kruskal and prim weights, docs(report): add plots.
- **Layout:** src/main/java (graph, algorithms, metrics, benchmark), src/test/java (tests), data/ (input graphs), results/results.csv, results/plots/, README.md, REPORT.md.

Design Quality Standards

Best-Case Design (Target)

- **One Graph Class:** All six algorithms run on the same graph type, so the benchmark is one loop.
- **Self-Checking Benchmark:** It compares the two MST weights, validates the topological order and prints the reference values - a broken algorithm fails the run instead of writing a nice number into the CSV.
- **Stack-Safe Traversal:** Depth is bounded by your own stack, and the measured maxDepth is in the CSV.
- **Reproducible Results:** The same seed and one command reproduce every number in your report.

Worst-Case Design (Avoid)

- **Recursive DFS:** Dies with StackOverflowError on 100 000 vertices, which is exactly the interesting size.
- **Kruskal Without DSU:** Checking for a cycle with a BFS on every edge turns O(E log E) into O(V·E).
- **Crash on a Disconnected Graph:** MST must return a forest, not an exception.

- **Counters Added Afterwards:** Numbers computed from a formula instead of counted inside the algorithm.
- **Conclusions From the Clock:** "Prim is faster" measured once on one laptop, with no counter to back it up.

Bonus Points (Extra +15%)

Task A: Tarjan's SCC (+10%) Implement Tarjan's algorithm, which finds all strongly connected components in a **single** DFS pass using index[], lowlink[] and a stack of active vertices. It must return the same components as Kosaraju on every test graph. Compare the edge scans of the two algorithms and explain in the report where the saving comes from.

Task B: Condensation & Critical Path (+5%) Build the condensation graph (every SCC becomes one vertex; the result is always a DAG), then find the **longest** path in it by relaxing edges in topological order with the sign flipped. Explain why the longest path is easy in a DAG and hard in a general graph, and where a critical path is used in scheduling.

Deliverables

1. **Source Code:** Maven or Gradle project with the graph, the six algorithms, metrics and JUnit 5 tests.
2. **Input Graphs:** At least three small graphs in data/, including one disconnected and one cyclic.
3. **results.csv:** All algorithms on all four generated graphs, with the columns listed above.
4. **Plots:** Three PNG charts in results/plots/.
5. **Report:** REPORT.md or PDF with the complexity table, the counter-vs-formula check, the two correctness arguments, the "which algorithm when" table and the plots.
6. **README:** How to build the project, run the tests and run the benchmark, plus your AI-use acknowledgement.

Grading Rubric

Criterion	Weight	What is evaluated	High-Performance Indicators
Graph Core & DSU	20%	Graph class, file input, DSU	Adjacency list; both DSU heuristics; counters inside the operations; edge cases handled.
MST: Kruskal & Prim	20%	Both MST algorithms	Kruskal uses DSU, Prim uses a priority queue; equal total weight; forest on a disconnected graph.
SCC & Topological Order	20%	Kosaraju and Kahn	Iterative DFS safe at 100 000 vertices; correct components; cycle detected instead of a wrong order.

Criterion	Weight	What is evaluated	High-Performance Indicators
Shortest Paths in DAGs	10%	Relaxation in topological order	One O(V + E) pass; super-source; negative weights handled; path restored.
Metrics & Report	20%	Counters, reference values, analysis	Reference values reproduced exactly; counter-vs-formula check; clear "which algorithm when" table.
Testing & Git	10%	JUnit 5 tests, README, history	Cross-checks against brute force and Bellman-Ford; feature branches; tag v1.0; builds with one command.